

Hvordan bruke secrets i applikasjonen

Disse guidene viser deg hvordan du kan hente secrets, pullsecrets, og virksomhetssertifikater fra et Key Vault og hvordan de kan brukes i applikasjonen din.

Bruke secrets

For at applikasjonen skal kunne hente secrets må det opprettes 2 ressurser:

- `SecretStore` - Brukes for å koble til et Key Vault
- `ExternalSecret` - Spesifiserer hvilke secrets som skal synkroniseres

Disse ressursene anbefales å legges sammen med de andre manifestene til applikasjonen og deployes på vanlig måte. Husk at hvert miljø har sitt eget Key Vault. Dersom applikasjon `guestbook` har 3 miljøer kan strukturen se slik ut:

```
applicationsets
├── prod
│   └── guestbook
│       ├── external-secret-guestbook.yaml
│       └── secret-store-guestbook.yaml
├── test
│   └── guestbook
│       ├── external-secret-guestbook.yaml
│       └── secret-store-guestbook.yaml
└── preprod
    └── guestbook
        ├── external-secret-guestbook.yaml
        └── secret-store-guestbook.yaml
```

Forutsetninger

- Det er opprettet Key Vault for applikasjonen. Se eventuelt [Opprette Key Vault](#).
- Secrets er lagt til i Key Vault. Se eventuelt [Administrere secrets](#).

Opprett secretstore

1. Kopier malen under til `applicationsets/<miljø>/<applikasjon>/secret-store-<applikasjon>.yaml`.

Mal

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: secret-store-<applikasjon>
spec:
  provider:
    azurekv:
      tenantId: "e6795081-6391-442e-9ab4-5e9ef74f18ea"
      vaultUrl: https://<keyvault>.vault.azure.net
      authSecretRef:
        clientId:
          name: keyvault-credentials
          key: client_id
        clientSecret:
          name: keyvault-credentials
          key: client_secret
```

Eksempel

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: secret-store-fosterhjem-api
spec:
  provider:
    azurekv:
      tenantId: "e6795081-6391-442e-9ab4-5e9ef74f18ea"
      vaultUrl: https://akt-fosthjem-api-prod.vault.azure.net
      authSecretRef:
        clientId:
          name: keyvault-credentials
          key: client_id
        clientSecret:
          name: keyvault-credentials
          key: client_secret
```

2. Fyll ut feltene:

- `applikasjon` - Navn på applikasjonen. Kan ofte hentes via template-parametere.
- `keyvault` - Det globalt unike navnet som refererer til Key Vault for dette miljøet. Følger som regel en variasjon av `<tenant>-<applikasjon>-<miljø>`.

3. Deploy.

Opprett externalsecret

1. Kopier malen under til `applicationsets/<miljø>/<applikasjon>/external-secret-<applikasjon>.yaml`.

Mal

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: external-secret-<applikasjon>
spec:
  refreshInterval: 24h0m0s # Oppdateringsfrekvens. Kan endres til
                             lavere frekvens, men det vil medføre høyere kostnad.
  secretStoreRef:
    kind: SecretStore
    name: secret-store-<applikasjon>
  target:
    name: secrets-<applikasjon> # Referansen for å aksessere secrets
                                i andre ressurser
    creationPolicy: Owner
    template:
      engineVersion: v2
      data: # Her setter du opp variabler med referanse til secret
            data slik at de kan brukes som miljøvariabler
#      ABC_TESTSECRET: "{{ .testSecret1 }}" # Henter
secret fra secret data og gjør den tilgjengelig som miljøvariabel
"ABC_TESTSECRET"
#      ABC_DENNEVARBASE64: "{{ .b64encoded | b64dec }}" # Henter
secret fra secret data, base64decoder tilgjengeliggjør den som
miljøvariabel "ABC_DENNEVARBASE64"
      data: # Her setter du opp variabler med referanser som henter
            secrets fra Azure KeyVault
#      - secretKey: testSecret1 # Navnet
secreten vil ha under target/template/data
#      remoteRef:
#      key: test-secret # Navnet
til secreten i Azure KeyVault
#      - secretKey: b64encoded # Navnet
secreten vil ha under target/template/data
#      remoteRef:
#      key: test-secret2 # Navnet
til secreten i Azure KeyVault
```

Eksempel

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: external-secret-fosterhjem-api
spec:
  refreshInterval: 24h0m0s
  secretStoreRef:
    kind: SecretStore
    name: secret-store-fosterhjem-api
  target:
    name: secrets-fosterhjem-api
    creationPolicy: Owner
    template:
      engineVersion: v2
      data:
```

```
mysecret: "{{ .mysecretkey }}"
data:
  - secretKey: mysecretkey
    remoteRef:
      key: mysecret
```

2. Fyll ut feltene:

- `applikasjon` - Navn på applikasjonen. Må være samme som brukes i `SecretStore` for at referanse til ressursen skal bli riktig.
- `secret` - Navn på secret som er lagt til i Key Vault for det spesifikke miljøet.
- **NB!** `refreshInterval` må ha med hours, minutes og seconds selv om minutes og seconds settes til 0. Dvs. man må bruke `24h0m0s` og **ikke** `24h`. Dersom man ikke har med minutes og seconds vil `ExternalSecret` prøve å synce hele tiden og dette vil medføre høy last på `ExternalSecret` operatoren. Dette kan videre medføre at uthenting av secrets går tregere for applikasjoner i flere tenants.

Helm template

Dersom helm benyttes for templating må det escapes:

```
mysecret: "{{ `{{ .mysecretkey }}` }}"
```

Se offisiell guide om [advanced templating](#) til `ExternalSecrets`.

3. Deploy.

Referer til secret

Det skal nå være generert en Kubernetes `Secret`. Denne kan refereres til i deployment:

```
containers:
  - envFrom:
    - secretRef:
        name: secrets-<applikasjon>
```

For referanse, se den offisielle dokumentasjonen til [SecretStore](#) og [ExternalSecret](#).

Bruke pullsecrets

Pull-secrets er per team og skal lagres i cluster-wide secrets. Se [dokumentasjon her](#)

For at applikasjonen skal kunne bruke pullsecrets til å hente container-images må det opprettes 2 ressurser:

- `githubregcred` - Henter pullsecret til Github Packages
- `dockerregcred` - Henter pullsecret til DockerHub

Forutsetninger

- Det er lagt til pullsecrets i Key Vaultet. Se eventuelt [Administrere secrets i Azure](#).

Opprett externalsecrets

1. Sett opp filen `applicationsets/<miljø>/namespace-secrets/githubregcred.yaml`:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: githubregcred
spec:
  refreshInterval: 24h0m0s
  secretStoreRef:
    kind: ClusterSecretStore
    name: <virksomhet>-team-<ditt team>-clusterwide-secret-store
  target:
    name: githubregcred
    creationPolicy: Owner
    template:
      engineVersion: v2
      type: kubernetes.io/dockerconfigjson
      data:
        .dockerconfigjson: |
          {
            "auths": {
              "ghcr.io": {
                "auth": "{{ printf "<team>-github-service-user:%s"
.GITHUB_SERVICE_USER_PAT | b64enc }}"
              }
            }
          }
      data:
        - secretKey: GITHUB_SERVICE_USER_PAT
          remoteRef:
            key: github-service-user-pat # navnet på nøkkelen din i
keyvault
```

2. Sett opp filen `applicationsets/<miljø>/namespace-secrets/dockerregcred.yaml` :

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: dockerregcred
spec:
  refreshInterval: 24h0m0s
  secretStoreRef:
    kind: ClusterSecretStore
    name: <virksomhet>-team-<ditt team>-clusterwide-secret-store
  target:
    name: dockerregcred
    creationPolicy: Owner
    template:
      engineVersion: v2
      type: kubernetes.io/dockercfg
      data:
        .dockercfg: |
          {
            "https://index.docker.io/v1/": {
              "auth": "{{ printf "<team>-github-service-user:%s"
.DOCKER_PAT | b64enc }}"
            }
          }
      data:
        - secretKey: DOCKER_PAT
          remoteRef:
            key: dockerhub-read-only-pat # navnet på nøkkelen din i
keyvault
```

Setup pullsecrets med Sealed secret

Hvis dere ikke har migrert til cluster-wide hemmeligheten, må dere bruke Sealed Secrets-oppsettet:

For at applikasjonen skal kunne bruke pullsecrets til å hente container-images må det opprettes 3 ressurser:

- `secretstore-pullsecrets` - Brukes for å koble til Key Vaultet med pullsecrets
- `githubregcred` - Henter pullsecret til Github Packages
- `dockerregcred` - Henter pullsecret til DockerHub

Forutsetninger

- Det er lagt til pullsecrets i Key Vaultet. Se eventuelt [Administrere secrets i Azure](#).

Opprett secretstore

1. Sett opp filen `applicationsets/<miljø>/namespace-secrets/secretstore-pullsecrets.yaml`:

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: secretstore-pullsecrets
spec:
  provider:
    azurekv:
      tenantId: "e6795081-6391-442e-9ab4-5e9ef74f18ea"
      vaultUrl: "https://<keyvault-navn>.vault.azure.net/"
      authSecretRef:
        clientId:
          name: keyvault-credentials
          key: client_id
        clientSecret:
          name: keyvault-credentials
          key: client_secret
```

2. Erstatt `<keyvault-navn>` med navn på Key Vault til pullsecrets.

Opprett externalsecrets1. Sett opp filen

`applicationsets/<miljø>/namespace-secrets/githubregcred.yaml` :

```
```yaml
apiVersion: external-secrets.io/v1beta1

kind: ExternalSecret
metadata:
```

```

 name: githubregcred
spec:
 refreshInterval: 24h0m0s
 secretStoreRef:
 kind: SecretStore
 name: secretstore-pullsecrets
 target:
 name: githubregcred
 creationPolicy: Owner
 template:
 engineVersion: v2
 type: kubernetes.io/dockerconfigjson
 data:
 .dockerconfigjson: |
 {
 "auths": {
 "ghcr.io": {
 "auth": "{{ .GITHUBREGCRED | b64enc }}"
 }
 }
 }
 data:
 - secretKey: GITHUBREGCRED
 remoteRef:
 key: githubregcred
...

```

1. Sett opp filen `applicationsets/<miljø>/namespace-secrets/dockerregcred.yaml`:

```

apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
 name: dockerregcred
spec:
 refreshInterval: 24h0m0s
 secretStoreRef:
 kind: SecretStore
 name: secretstore-pullsecrets
 target:
 name: dockerregcred
 creationPolicy: Owner
 template:
 engineVersion: v2
 type: kubernetes.io/dockercfg
 data:
 .dockercfg: |
 {
 "https://index.docker.io/v1/": {
 "auth": "{{ .DOCKERREGCRED | b64enc }}"
 }
 }
 data:
 - secretKey: DOCKERREGCRED
 remoteRef:
 key: dockerregcred

```

## Referer til pullsecrets

githubregcred

```
imagePullSecrets:
- name: githubregcred
```

dockerregcred

```
imagePullSecrets:
- name: dockerregcred
```

Applikasjonen skal nå kunne hente container-images fra GitHub Packages og DockerHub.

Les eventuelt mer om "[Pull an Image from a Private Registry](#)" i den offisielle dokumentasjonen til Kubernetes.

## Bruke virksomhetssertifikater

### WIP

Fram til Team Kjøremiljø har automatisering opprettelsen av disse filene må denne prosessen gjøres manuelt av teamene. Dokumentasjonen vil bli oppdatert når dette ikke lenger er nødvendig.

Credentials for tilgangen til Key Vaults ( `SealedSecrets` ) for sertifikatene er de samme som for andre Key Vaults, så det trengs ikke å legge til dette lenger.

Det finnes 2 Key Vault for virksomhetssertifikater, `uke-vsert-test` og `uke-vsert-prod`. Det må opprettes `SecretStore` og `ExternalSecret` for hvert miljø hvor dette brukes.

1. Kopier innhold til en fil under `applicationsets/<miljø>/namespace-secrets` som heter `secretstore-virksomhetssertifikat.yaml`:

Prod

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
 name: secretstore-virksomhetssertifikat
spec:
 provider:
 azurekv:
```

```
tenantId: "e6795081-6391-442e-9ab4-5e9ef74f18ea"
vaultUrl: https://uke-vsrt-prod.vault.azure.net
authSecretRef:
 clientId:
 name: keyvault-credentials
 key: client_id
 clientSecret:
 name: keyvault-credentials
 key: client_secret
```

### Test/preprod

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
 name: secretstore-virksomhetssertifikat
spec:
 provider:
 azurekv:
 tenantId: "e6795081-6391-442e-9ab4-5e9ef74f18ea"
 vaultUrl: https://uke-vsrt-test.vault.azure.net
 authSecretRef:
 clientId:
 name: keyvault-credentials
 key: client_id
 clientSecret:
 name: keyvault-credentials
 key: client_secret
```

2. Kopier innhold til en fil under `applicationsets/<miljø>/namespace-secrets` med navn `externalsecret-virksomhetssertifikat.yaml`.

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
 name: virksomhetssertifikat
spec:
 refreshInterval: 24h0m0s
 secretStoreRef:
 kind: SecretStore
 name: secretstore-virksomhetssertifikat
 target:
 name: virksomhetssertifikat
 creationPolicy: Owner
 template:
 engineVersion: v2
 data:
 uke-virksomhetssertifikat-authentication.p12: "{{
 .virksomhetssertifikatUkeAuthentication | b64dec }}" # Dersom du har
 behov for autentiseringssertifikatet
 uke-virksomhetssertifikat-signing.p12: "{{
 .virksomhetssertifikatUkeSigning | b64dec }}" # Dersom du har behov
 for signeringssertifikatet
 VIRKSOMHETSSERTIFIKAT_UKE_PASSWORD: "{{
 .virksomhetssertifikatUkePassword }}"
 data:
```

```
- secretKey: virksomhetssertifikatUkeAuthentication
 remoteRef:
 key: virksomhetssertifikat-maskinporten-uke-authentication
- secretKey: virksomhetssertifikatUkeSigning
 remoteRef:
 key: virksomhetssertifikat-maskinporten-uke-signing
- secretKey: virksomhetssertifikatUkePassword
 remoteRef:
 key: virksomhetssertifikat-maskinporten-uke-password
```

3. Deploy `ExternalSecret` og `SecretStore`.

Applikasjonen kan nå bruke en Kubernetes Secret kalt `virksomhetssertifikat`.